Participant Manual (Official Edition)

GaugeForge

Discord Community
GaugeForge Agentic Quantum Coding Challenge's official Q&A channel. Participants must join.
https://discord.gg/bSyjG6sz5

Contents / 目录

1. Event overview and timeline 1
2. Accounts and contact 2
3. Evaluation types and attempts 2
4. Build a harness 3
5. Submission and status 6
6. Scoring and leaderboard 7
7. Help 8
8. Frequently asked questions and troubleshooting 8

This manual explains how to enter the QIQC Challenge, build a harness, submit it, understand evaluation results and handle common failures. All times are Asia/Singapore (UTC+8).

1. Event overview and timeline

The QIQC Challenge evaluates an **agent harness**: an auditable code package that drives a language model through scientific tasks in Quantum-Harbor, a simulated quantum laboratory. You submit harness code, not answers or prompts. Official evaluations use the organizer's model, task edition, verifiers and per-task time limits in an isolated environment. The platform record is the official record.

Time (UTC+8)	What happens
6 November 2026, 20:00	Infrastructure test opens: build and upload a harness, then run tests on the 3 infrastructure cases; it is unranked.
8 November 2026, 00:00	Infrastructure test closes and an 8-hour quiet period begins; uploads and new evaluations close.
8 November 2026, 08:00	Competition opens: Public tests and Full evaluation become available and the leaderboard begins publishing results.

Time (UTC+8)	What happens
10 November 2026, 21:00	Competition closes; no new uploads or evaluations are accepted, while accepted evaluations run to natural completion.
11 November 2026, 21:00	Formal appeals close.
12 November 2026	Invited teams give final presentations.

When the quiet period starts, evaluations still waiting in the queue and not yet started are cancelled and the team sees a notice. Evaluations already running are not interrupted and finish naturally.

The competition model is provided through the unified model gateway. **The Codex + Astra(xhigh) baseline will be published with the leaderboard when the official competition starts.**

2. Accounts and contact

1. Sign in to Submit with the team account issued by the organizers.
2. On the first sign-in, replace the temporary password with a new password of 12–200 characters.
3. If you forget the password, email **qiqc@gauge-forge.com** from the team account email and request a reset. Never send a password, model key, token or private trajectory in email.
4. After too many failed sign-ins, wait 15 minutes before trying again.

Each team may use only its own account. Sharing an account, signing in for another team or submitting for another team violates the rules.

3. Evaluation types and attempts

Type	Tasks	Attempts per team	Leaderboard	Active submission
Infrastructure test	3 infrastructure cases	10	No	Yes
Public test	5 public tasks	3	No	Yes
Full evaluation	39 normal tasks	1	Yes	Yes
Finalist evaluation	7 competition tasks	1	No; supporting evidence for judges	No

Infrastructure tests, Public tests and Full evaluation have independent participant quotas.

The 7 competition tasks form the finalist evaluation and apply only to teams that place in the top five of the Full evaluation. They are not shown as a selectable evaluation and do not consume a team quota. The event backend automatically evaluates the submission version that achieved each team's best Full evaluation result; teams do not submit it themselves, and the result is supporting evidence for the judges' final ranking decision.

Rules for participant-visible evaluations:

- A team can have only one evaluation in progress. A queued, running or staff-resolved evaluation blocks a new request.

- An attempt is charged only when an evaluation is accepted. Upload, format checks and automated review do not charge an attempt. Rejection, a closed stage, exhausted quota or an existing evaluation does not charge one.
- A network interruption or page refresh does not create a second evaluation. Refresh Evaluation history and confirm the original state before acting.
- Cancelling while a request is queued does not refund an attempt; once accepted, cancelling still consumes it.
- If a run fails because of the platform, task environment, model service or result storage, staff handle or rerun the original request without charging another team attempt. Harness failures are recorded as task failures.
- Each infrastructure case has a 3-minute limit. Public tests and Full evaluation use each task's original task time limit.
- A team may keep at most 25 uploaded versions.

4. Build a harness

4.1 Start from a template

Open Submit and download one template: Codex, DeepSeek Harness or Pi. Each template runs its native command-line Agent inside the task container; modify it as the basis of your submission. Submitting an unmodified starter template is automatically rejected before execution. Extensive copying, or directly adopting an open-source harness framework that you did not develop, can invalidate the result after review; the event judging panel has final authority.

Template	`runtime`	Supported runtime versions	Model interface
Codex	`codex`	`@openai/codex` 0.166.0, 0.165.0, 0.164.0, 0.163.0, 0.162.1, 0.162.0, 0.144.4	Responses
DeepSeek Harness	`deepseek`	`@deepseek-ai/dsh` 0.2.2-rc.1, 0.2.1-rc.5, 0.2.1-rc.4, 0.2.1-rc.3, 0.2.1-rc.2	Chat Completions (also Responses)
Pi	`pi`	`@earendil-works/pi-coding-agent` 1.5.0, 1.4.0, 1.3.0, 1.2.0, 1.1.0, 1.0.4	Chat Completions (also Responses)

The platform accepts the supported versions listed above. Runtime binaries are selected from an organizer-managed execution snapshot; participants do not install packages in the task container. Legacy manifests may declare any version in the list, and the parser normalizes them to the snapshot used for that run.

4.2 Package layout

Submit one ZIP whose root contains exactly these three entries:

```
harness.zip
├── harness.json
```

```
├── adapter/
│   └── run.sh
└── harness/
    ├── your_agent_code.py
    └── ...
```

`adapter/` must contain an entry script ending in `.sh`, and `harness/` must contain at least one file. Do not wrap the whole project in an additional top-level directory; archive the project contents.

Limits: 20 MiB compressed, 100 MiB extracted, 1,000 entries, and a maximum compression ratio of 100:1 per file. Symlinks, special files, hidden files (except `.gitignore`), absolute paths, `..` path segments, encrypted entries and other compression methods are forbidden. Put all dependencies in `harness/`; the runtime cannot download or install them.

4.3 harness.json

The file must contain the required runtime entry:

```
{"runtime": "codex"}
```

The complete optional-field specification is below; fields cannot be added or changed:

```
{
  "runtime": "pi",
  "entrypoint": "adapter/run.sh",
  "name": "Planner and verifier loop",
  "description": "Pi agent with a planning prompt and a self-check step."
}
```

Field	Required	Allowed values	Default
`runtime`	Yes	`codex`, `deepseek` or `pi`	None
`entrypoint`	No	A `.sh` path inside `adapter/`	`adapter/run.sh`
`name`	No	Free text, 1–80 characters	Generated from runtime
`description`	No	Free text, up to 500 characters	Empty

The platform derives the runtime package, supported version window, model interface, task transport and trajectory format from `runtime`. Do not write versions, credentials, tokens, model names, time budgets or task selections in `harness.json`. The platform numbers versions in upload order, such as `v1` and `v2`. Uploading exactly the same bytes again returns the existing version and refuses to run it.

4.4 Runtime inputs

The platform extracts the ZIP in the task working directory and runs `sh <entrypoint>`. It provides these environment variables:

Variable	Meaning
`QIQC_TASK_INPUT`	Path to the task description
`QIQC_MODEL_URL`	Base URL of the competition model gateway
`QIQC_MODEL_TOKEN`	Short-lived token for this run
`QIQC_MODEL`	Model ID
`QIQC_MODEL_PROTOCOL`	`responses` or `chat/completions`
`QIQC_AGENT_TIMEOUT`	Agent time budget for this task, in seconds
`QIQC_MCP_SERVERS`	JSON list of task tool servers
`QIQC_OUTPUT`	Path for the final message or summary
`QIQC_TRAJECTORY`	Path for the Agent event log in JSON Lines

Never hard-code paths, URLs or tokens. Send the token only to `QIQC_MODEL_URL`.

4.5 Results, verifier and trajectories

When a task ends, the platform first collects the answer artifacts, execution state, Agent time and controlled logs from the task container, then passes them to that task's verifier. The verifier checks the files, fields, values, tolerances or experimental results defined by the task and returns a binary pass/fail result with an auditable runtime.

The platform retains the complete internal rollout, while teams can see only the public trajectories allowed by the rules. During the competition, Evaluation history exposes filtered trajectories only for the three Public tests, with five public-task traces per test. After a Public test completes, click View traces for that record. A panel opens below the record with a folder labelled Public test #1, #2 or #3 and five files. If the files are not available immediately after completion, click the rotating Refresh control. The downloads are filtered trajectories only; they do not include the full rollout, hidden-task logs, scoring material, credentials or another team's data.

4.6 Prohibited conduct

- Put API keys, provider keys, private keys, passwords or `.env` files in the ZIP.
- During infrastructure testing, access only the model gateway and the task tool servers. Official evaluations follow the task's original network policy, but may not read backend credentials, private scoring material or another team's data.
- Read hidden tasks, reference answers, verifiers, scoring code, cloud metadata, Docker sockets, host paths or another team's data.
- Change network policy, tamper with scoring, interfere with the platform or leave a resource-consuming background service running.
- If the system detects a submission with data theft or a risk of disrupting normal event operation, and staff confirm it, the matter may be pursued under applicable law.

The organizers may inspect package contents, logs, trajectories and network events. Any violation directly invalidates the result and can cancel eligibility.

4.7 Local test

You need Docker, uv, git and Python 3.11 or newer; on Windows, use Linux containers through WSL2. From the root of the Quantum-Harbor source supplied with the event:

```
uv tool install 'harbor==0.23.0'
cd Quantum-Harbor
git checkout 1f03d78e959cf02469568250f61f9af88a53146f
uv sync
docker build -t qiqcbench-agent docker/agent/
uv run python tools/build_qsim_images.py \
  --tasks time_budgeted_hamlearn_10q
harbor run -p harbor_tasks/time_budgeted_hamlearn_10q -a nop
harbor view jobs
```

`nop` performs no science, so reward 0 is expected; this checks installation only. For an entry-script smoke test, set temporary local variables. Your own key is for local use only and must never enter the ZIP:

```
export QIQC_TASK_INPUT=$PWD/task.txt QIQC_MODEL=your-model
export QIQC_MODEL_URL=YOUR_MODEL_GATEWAY_URL
export QIQC_MODEL_TOKEN=your-own-key QIQC_AGENT_TIMEOUT=600
export QIQC_MODEL_PROTOCOL=chat/completions
export QIQC_OUTPUT=/tmp/out/final.txt
export QIQC_TRAJECTORY=/tmp/out/trajectory.jsonl
sh adapter/run.sh
```

Local models, networks and hardware differ from the event environment. Use the infrastructure test to confirm that the package runs on the platform.

4.8 Pre-upload checklist

- The ZIP root contains only `harness.json`, `adapter/` and `harness/`, and is under 20 MiB.
- `runtime` is valid, and the entry script exists and ends in `.sh`.
- No credentials, `.env` files, hidden files, links, caches or unrelated large files are present.
- The harness writes the answer artifacts and trajectory required by the task.
- At least one public task has been tested locally.

5. Submission and status

5.1 Upload

1. Sign in to Submit.
2. Select an evaluation type that is open.
3. Drop the ZIP or choose it in the upload area.
4. Click **Upload and evaluate** once.

The platform scans the archive, stores the immutable bytes and performs automated review. After approval, the evaluation is accepted and queued automatically, and one attempt is charged. You may close the page after the upload; the evaluation continues on the server. Sign in again to check it. When asking for help, provide the version SHA-256 and evaluation ID, never a private trajectory or key.

5.2 Status reference

Status	Meaning	What you should do
Uploaded, automated review	The ZIP was received and is being checked; no attempt charged.	Wait.
Under review	The version is waiting for automated review.	Wait; do not upload again.
Rejected / Review rejected	Format or security review failed; no attempt charged.	Read the reason, modify and upload a new version.
Not accepted	Review passed, but the stage is closed, quota is exhausted or another evaluation is active; no attempt charged.	Read the reason.
Approved / Accepted	Review passed and the evaluation was accepted; one attempt charged.	Save the evaluation ID.
Queued	Waiting for task slots.	Wait.
Running	Tasks are executing; partial scores are not shown.	Wait.
Cancelled	The evaluation was cancelled; an accepted attempt is not refunded.	No action.
Needs attention	A run could not finish and staff are checking it; no score is published yet.	Save the evaluation ID; do not resubmit automatically.
Completed	All tasks ended and the pass count and mean Agent time are shown.	Review the result; Public tests may show public trajectories.

If you believe the system misclassified your submission, email **qiqc@gauge-forge.com** with the team name, evaluation ID or version SHA-256, time, page and complete error text. Do not upload repeatedly to demonstrate the issue.

6. Scoring and leaderboard

- Each normal task receives a binary verifier result: pass (1) or fail (0).
- Full evaluation covers only the 39 regular tasks. Each team has one Full evaluation opportunity, and that result enters the leaderboard.
- Infrastructure tests and all three Public tests are excluded from the leaderboard.
- **Leaderboard data is ordered as follows:** teams are first compared by the number of the 39 tasks passed; ties are then compared by mean Agent runtime per task, with shorter time ranked higher. Queueing, preparation, scoring and upload time are excluded from Agent runtime.
- **First above baseline:** by evaluation acceptance time, the first team whose valid Full evaluation exceeds the Codex + Astra(xhigh) baseline wins the challenge award. The comparison uses the 39 normal tasks: more passes, or the same passes with a lower mean runtime. The 7 competition tasks are excluded.

- The leaderboard is a real-time reference only; final standings remain subject to the appeal period and judging review.
- **Top-five selection:** teams that qualify for the top five on the leaderboard receive an additional backend evaluation on the 7 competition tasks to produce an ELO ranking as supporting evidence for judging. At least three judges without conflicts of interest score each team. Judges use scientific reliability (40%), framework design (30%), generalizability (20%) and clarity (10%).
- **Demo day:** each finalist team presents one successful and one failed trajectory, with an 8-minute demonstration and 7 minutes for questions.
- **Prizes:**
  - Champion: USD 1,500
  - Second place: USD 700
  - Third place: USD 700
  - Fourth place: USD 500
  - Fifth place: USD 500
  - The USD 15,000 challenger award is granted only to the first rigorously reviewed, eligible team whose leaderboard result exceeds the GPT-6 Astra baseline.

7. Help

If you encounter a problem, email **qiqc@gauge-forge.com** from the team account email. Include the team name, evaluation ID or version SHA-256, time (UTC+8), page, complete error text, expected result and actual result. Do not attach passwords, API keys, tokens, hidden-task content or complete private trajectories. If you see another team's data, stop viewing, downloading or sharing it and report it privately by email.

Each team may file one formal appeal. Send it by 21:00 on 11 November 2026 (UTC+8), mark the subject **Formal appeal**, and include the evaluation ID, disputed rule, expected result, actual result and public evidence.

8. Frequently asked questions and troubleshooting

Problem 1: `harness.json: "runtime" is required`

Likely cause: `runtime` is missing or misspelled.

Fix: Add `"runtime": "codex"`, `"deepseek"` or `"pi"`, and validate the JSON.

Problem 2: The entrypoint cannot be executed or is missing

Likely cause: The entrypoint is outside `adapter/`, does not end in `.sh`, the ZIP has an extra top-level directory, or the script uses a local absolute path.

Fix: Inspect the ZIP tree, use `adapter/run.sh` as the default entrypoint, and use the supplied environment variables and relative paths.

Problem 3: `Rejected / Review rejected` after upload

Likely cause: Credentials, hidden files, symlinks, special files, size or entry limits, or unknown fields in `harness.json`.

Fix: Apply the review reason and create a new ZIP. Uploading identical bytes again returns the existing version and refuses to run it.

Problem 4: No evaluation type is available

Likely cause: The stage is closed, another evaluation is active, or the quota for that type is exhausted.

Fix: Read the Event timeline, current status and remaining attempts; do not click repeatedly to create another request.

Problem 5: The evaluation remains `Queued`

Likely cause: Task slots are busy or earlier queue entries are ahead of it.

Fix: Wait for dispatch. The Ahead value counts task dispatches waiting before yours. Cancelling a queued accepted request consumes its attempt.

Problem 6: The status is `Needs attention`

Likely cause: A task, model service, execution container or result store could not complete automatically.

Fix: Keep the evaluation ID and do not resubmit automatically. Staff will investigate; a confirmed platform failure is handled under the original request without an extra attempt.

Problem 7: The result has no downloadable trajectory

Likely cause: Only a completed Public test exposes public trajectories. Full evaluation, hidden tasks and finalist evaluation never expose them.

Fix: Confirm that the record is a Public test and all five tasks are complete. Click **View traces**, then **Refresh**. Email staff if the files still do not appear.

Problem 8: Only some Public test files appear

Likely cause: The result or R2 trajectory index is still being read back, and the page has not refreshed.

Fix: Click the rotating refresh control in the trace panel. A complete Public test must expose five filtered files; report the evaluation ID by email if it remains incomplete.

Problem 9: `QIQC_OUTPUT` or a required answer artifact is missing

Likely cause: The entry script exited early, wrote the answer only to stdout, ignored the task's artifact path or left work in a background process.

Fix: Follow the task's artifact paths and fields, verify the file locally and keep the entry script alive until the Agent has completed.

Problem 10: A model request fails, times out or is temporarily unavailable

Likely cause: Temporary provider capacity, an upstream error or an incompatible request protocol.

Fix: Never put the token in logs. Use `QIQC_MODEL_URL`, `QIQC_MODEL_TOKEN` and `QIQC_MODEL_PROTOCOL` as supplied by the platform. Staff investigate platform failures; fix harness errors in your own package.

Problem 11: Local and platform results differ

Likely cause: Local model, dependencies, network, resources and containers differ from the event environment.

Fix: Treat local testing as a script and artifact check. The platform's infrastructure and official evaluations are authoritative.

Problem 12: You are worried about charging an attempt twice

Likely cause: A refresh made the original request look absent and the upload or confirmation button was clicked again.

Fix: Check Evaluation history and the version list first. Attempts are charged only when accepted; retrying the same upload and request does not create a second evaluation.